



UNIVERSIDADE FEDERAL
DE ALAGOAS

Controle determinístico para CNC baseado em STM32L475 com DDA de alta frequência

Valdir Dias Silva Junior

26 de novembro de 2025



Universidade
Federal de
Alagoas



Roteiro

O

- ▶ Protocolo de comunicação
- ▶ Arquitetura de hardware
- ▶ Arquitetura de software
- ▶ Controle de movimento
- ▶ Resultados e conclusão

Sumário

1 Protocolo de comunicação

► Protocolo de comunicação

► Arquitetura de hardware

► Arquitetura de software

► Controle de movimento

► Resultados e conclusão

Visão geral do enlace

1 Protocolo de comunicação

- Arquitetura híbrida: Raspberry Pi como mestre de alto nível e STM32L475 como controlador em tempo real.
- Comunicação principal via SPI1 (STM32 em modo escravo com DMA circular).
- USART1 usada como console de depuração, telemetria e logs de eventos.
- Drivers TMC5160 configurados por SPI dedicado e monitorados por registradores de status.

Quadro SPI – Raspberry Pi ↔ STM32

1 Protocolo de comunicação

- Quadros de **tamanho fixo**: 42 bytes em modo *full-duplex*.
- Estrutura típica:
 - **SOF** (0xAB), **CMD**, **FLAGS**, **LEN**.
 - **PAYLOAD** com até 34 bytes úteis.
 - **CRC** de 16 bits sobre os bytes 0..37.
 - **EOF** (0x54) como marcador de fim de quadro.
- Quando não há resposta pronta, o escravo devolve padrão 0xA5 em todo o payload.
- Protocolo baseado em *poll*: o mestre envia pedidos periódicos até receber uma resposta válida.

Serviços SPI: LED e Movimento

1 Protocolo de comunicação

Serviço LED (CMD = 0x10)

- Requisição: identifica o LED e o modo (OFF, ON, TOGGLE).
- Resposta: campo STATUS (OK/erro) e LED_STATE final.
- Exemplo simples para validar integridade do enlace e CRC.

Serviço Movimento (CMD = 0x20)

- Requisição recebe AXIS_MASK, deslocamentos relativos em passos (X,Y,Z) e FEED em passos/s.
- Opcionalmente, parâmetro de *jerk* para suavizar aceleração.
- Resposta indica STATUS (aceito/ocupado/erro) e QUEUE_DEPTH da fila de movimentos.

SPI Raspberry Pi ↔ TMC5160

1 Protocolo de comunicação

- Transações de 40 bits (5 bytes): 1 byte de endereço + 4 bytes de dados.
- Bit mais significativo do endereço define leitura (1) ou escrita (0).
- Latência de uma transação: dados lidos em SDO correspondem ao comando anterior.
- Utilizado para:
 - Programar microstepping, correntes, modo de *chopper* e interpolação (*MicroPlyer*).
 - Ler DRV_STATUS e outros registradores de diagnóstico.

Fluxo do comando `cnc-cli`

1 Protocolo de comunicação

- Cliente em Python (`cnc_cli.py`) executa uma sequência declarativa em JSON.
- Passos incluem:
 - Aplicação de *patches* TMC5160 (corrente, microsteps, interpolação).
 - Atualização de ganhos PID inteiros por eixo.
 - Enfileiramento de movimentos relativos para o STM32.
- Cada comando é validado contra limites de `application.cfg`; violações abortam a sequência.
- Durante o movimento, o cliente monitora erros graves do TMC5160 e aciona medidas de segurança.

Sumário

2 Arquitetura de hardware

- ▶ Protocolo de comunicação
- ▶ **Arquitetura de hardware**
- ▶ Arquitetura de software
- ▶ Controle de movimento
- ▶ Resultados e conclusão

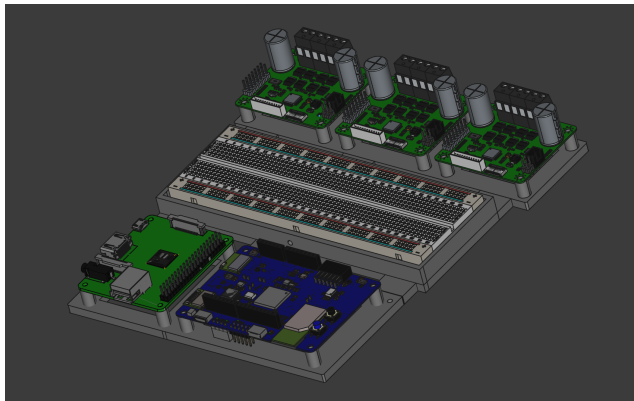
Blocos de hardware

2 Arquitetura de hardware

- **Lógica:** placa STM32L475 operando a 3.3 V, com interfaces SPI, USART e GPIO.
- **Drivers de eixo:** módulos com TMC5160 comandados por sinais STEP/DIR/EN.
- **Realimentação:** encoder óptico incremental TMCS-28 (ABN, TTL 5 V) acoplado ao eixo.
- **Alimentação:** trilha de 5 V para lógica/sensores e alimentação separada filtrada para potência.
- **Segurança:** entradas de E-STOP e sensores de fim de curso integradas ao NVIC com alta prioridade.

Layout do controlador CNC

2 Arquitetura de hardware



Modelo 3D do controlador CNC com placa STM32, interface para Raspberry Pi, base de prototipagem e módulos de potência TMC5160.

Encoder óptico TMCS-28

2 Arquitetura de hardware

- Encoder incremental TMCS-28-10k com resolução efetiva de 40 000 contagens por volta.
- Saídas A/B/N em TTL 5 V, condicionadas para 3.3 V antes de chegar ao STM32.
- Canais A/B ligados a TIM3, LPTIM1 e TIM5 em modo *encoder*, liberando a CPU de interrupções.
- Canal N utilizado como referência de índice na rotina de *homing*.

Sumário

3 Arquitetura de software

- ▶ Protocolo de comunicação
- ▶ Arquitetura de hardware
- ▶ **Arquitetura de software**
- ▶ Controle de movimento
- ▶ Resultados e conclusão

Camadas do firmware

3 Arquitetura de software

- **Core:**
 - Código gerado pelo STM32CubeMX (HAL, inicialização de clock, GPIO, timers, SPI, USART).
- **App:**
 - Laço principal `app_poll`, agendador de serviços e integração com filas SPI/USART.
- **Services:**
 - Gerador de passos, controle PID, serviço de homing, serviço de movimento e roteador de mensagens.

Pipeline SPI no firmware

3 Arquitetura de software

- SPI1 configurado como escravo com DMA circular recebendo quadros de 42 bytes.
- Cada *poll* do mestre:
 - Congela temporariamente o DMA para permitir processamento de comando em `app_poll`.
 - Se houver resposta pronta, o payload é promovido para o buffer ativo do DMA.
- Caso contrário, o mestre recebe quadro com padrão 0xA5 e repete o *poll*.
- Parâmetro `APP_SPI_RESTART_DEFER_MAX` ajusta o número máximo de tentativas antes de *fallback*.

Roteiro incremental de inicialização

3 Arquitetura de software

- Configuração do clock principal para 80 MHz.
- Definição de prioridades no NVIC: E-STOP, TIM6, SPI1/DMA, TIM7, USART1.
- Calibração dos temporizadores:
 - TIM6 @ 50 kHz para geração de *STEP*.
 - TIM7 @ 1 kHz para rampas, leitura de encoders e controle.
- Inicialização do SPI1 com DMA circular e integração com o serviço SPI.
- Integração da USART1 como console de log não bloqueante.



Sumário

4 Controle de movimento

- ▶ Protocolo de comunicação
- ▶ Arquitetura de hardware
- ▶ Arquitetura de software
- ▶ **Controle de movimento**
- ▶ Resultados e conclusão

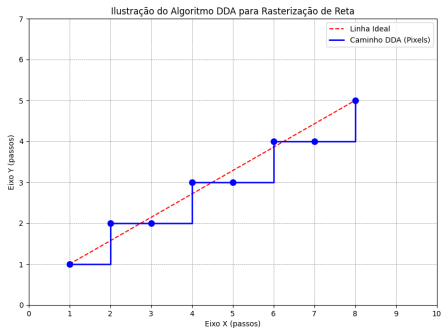
Temporizadores e DDA

4 Controle de movimento

- TIM6 configurado com $PSC = 79$ e $ARR = 19$, resultando em 50 kHz.
- Cada tick de $20 \mu s$ alimenta o DDA em ponto fixo para geração de *STEP*.
- TIM7 configurado para 1 kHz, responsável por:
 - Cálculo de rampas trapezoidais (aceleração/desaceleração).
 - Leitura dos contadores de encoder e atualização dos erros.
 - Aplicação do controle PI/PD de posição/velocidade.

DDA: geração de passos determinística

4 Controle de movimento



O DDA acumula erro fracionário a cada tick de TIM6 e gera pulsos *STEP* quando o limiar é ultrapassado, aproximando trajetórias contínuas por incrementos discretos sincronizados entre eixos.

Compatibilidade com o TMC5160 (STEP/DIR)

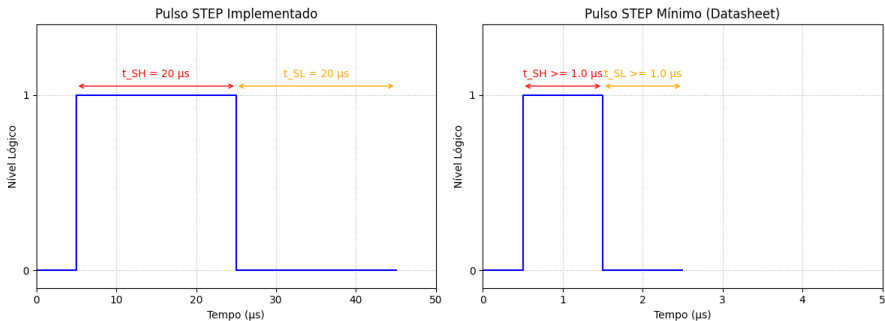
4 Controle de movimento

- Datasheet do TMC5160 impõe tempos mínimos de pulso na ordem de dezenas de ns.
- Projeto utiliza:
 - $t_{SH} = t_{SL} = 20 \mu s$ para largura alta e baixa de *STEP*.
 - $t_{DSU} = 20 \mu s$ entre mudança em *DIR* e próximo *STEP*.
 - Tempo de assentamento de *ENABLE* de $40 \mu s$.
- Margens de segurança de várias ordens de grandeza em relação às especificações.

Comparação de pulsos STEP

4 Controle de movimento

Comparação do Pulso de STEP: Firmware vs. Datasheet



Pulso de *STEP* implementado no firmware (esquerda) comparado ao pulso mínimo requerido pelo TMC5160 (direita), evidenciando ampla margem temporal.

Malhas de posição e velocidade

4 Controle de movimento

- Encoders em modo quadratura fornecem posição incremental precisa para cada eixo.
- Laço em 1 kHz calcula:
 - Erro de posição entre referência (trajetória do DDA) e leitura do encoder.
 - Erro de velocidade derivado das contagens por janela de tempo.
- Controladores PI/PD com ganhos inteiros garantem rastreamento estável e limitam sobre-elevação.
- Supervisão de sincronização cruzada entre eixos (*cross-coupled control*).

Modelagem discreta posição → velocidade

4 Controle de movimento

- Saída medida: posição incremental em passos DDA, a 1 kHz, a partir dos contadores de encoder.
- Em cada amostra, o erro de posição alimenta um controlador PI/PD discreto que ajusta a velocidade de referência do DDA.
- O termo derivativo é filtrado exponencialmente para reduzir ruído de quantização e vibrações rápidas.
- Assim, a posição medida no tempo discreto é convertida em correções de frequência de STEP (*steps/s*) aplicadas ao DDA.

Da física à função de transferência

4 Controle de movimento

- Modelo físico (2ª lei de Newton para rotação):

$$J \frac{d\omega(t)}{dt} + B\omega(t) = T_{\text{motor}}(t).$$

- Driver TMC5160 converte frequência de STEP em torque aproximado:

$$T_{\text{motor}}(t) \approx K_{\text{sist}} f(t).$$

- Substituindo e normalizando por B :

$$\tau = \frac{J}{B}, \quad K = \frac{K_{\text{sist}}}{B}, \quad \tau \frac{d\omega(t)}{dt} + \omega(t) = K f(t).$$

- Aplicando a transformada de Laplace (condições iniciais nulas):

$$G(s) = \frac{\Omega(s)}{F(s)} = \frac{K}{\tau s + 1}.$$

Modelo FOPDT da malha de velocidade

4 Controle de movimento

- Malha de velocidade aproximada por modelo FOPDT em malha aberta:

$$G_v(s) = \frac{K e^{-Ls}}{\tau s + 1}.$$

- Degrau de comando U gera resposta teórica

$$v(t) = K U (1 - e^{-(t-L)/\tau}) H(t - L),$$

derivada via transformada de Laplace.

- Parâmetros K , L e τ extraídos de degraus medidos via telemetria (patamar de velocidade, atraso e constante de tempo).
- Catálogo (K, L, τ) organizado por eixo e microstep alimenta a síntese dos ganhos do controlador.

Síntese PD via Ziegler–Nichols

4 Controle de movimento

- Com o modelo FOPDT identificado, aplica-se Ziegler–Nichols (curva de reação) para um regulador PD ($K_i = 0$):

$$K_p = 1,2 \frac{\tau}{K L}, \quad T_d = 0,5 L, \quad K_d = K_p T_d.$$

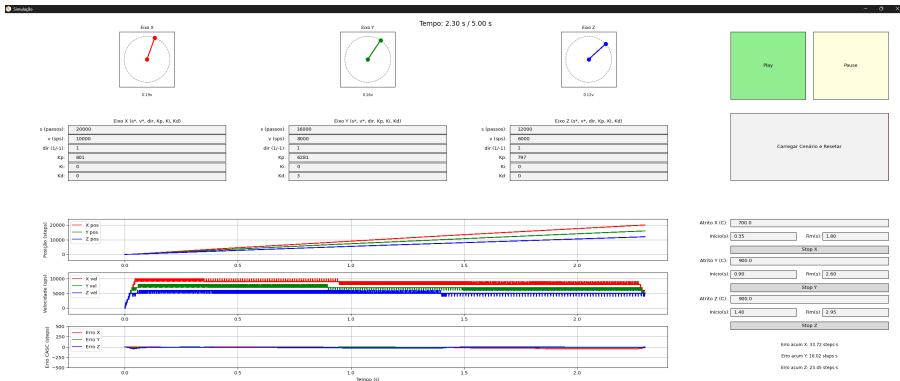
- Ganhos contínuos (K_p, K_d) convertidos para inteiros com fator de escala $S = 256$:

$$k_p = \text{round}(S K_p), \quad k_d = \text{round}(S K_d).$$

- Catálogo de ganhos inteiros por eixo e microstep compartilhado entre o firmware STM32 e o simulador em Python.
- A mesma estrutura de ganhos é usada na validação numérica e nos experimentos, garantindo consistência entre modelo e implementação.

Validação do modelo FOPDT

4 Controle de movimento



Sobreposição entre velocidade medida e resposta teórica do modelo FOPDT, mostrando boa aderência na faixa de operação de interesse para os três eixos.



Sumário

5 Resultados e conclusão

- ▶ Protocolo de comunicação
- ▶ Arquitetura de hardware
- ▶ Arquitetura de software
- ▶ Controle de movimento
- ▶ Resultados e conclusão

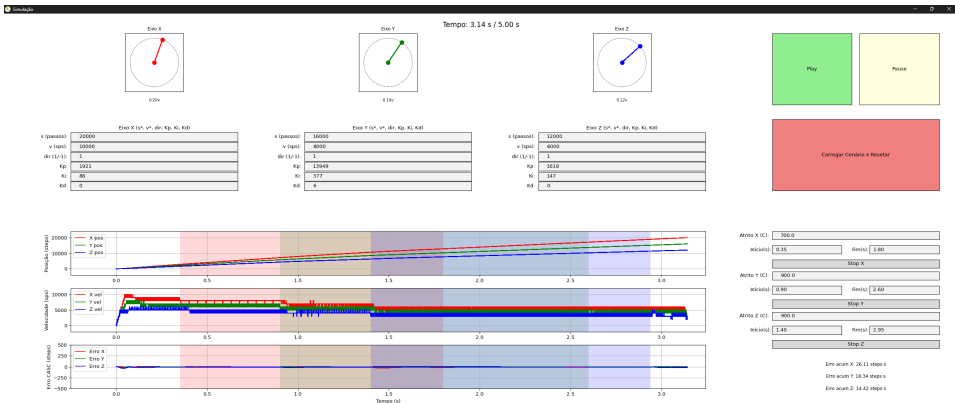
Desempenho dos serviços principais

5 Resultados e conclusão

- Gerador de passos (TIM6) opera próximo de 50 kHz com variação pequena entre ciclos.
- Laço de controle (TIM7) apresenta *jitter* na ordem de poucos μ s.
- Pipeline SPI confirma necessidade de 2–3 *polls* para entrega de respostas completas.
- Otimização de promoção direta de payload ao buffer de DMA reduziu o tempo médio entre *polls*.

Comparação simulação vs experimento

5 Resultados e conclusão



Comparação entre velocidades de passo medidas e simuladas sob carga: boa coerência geral, com desvios explicados por atrito e assimetrias de carga entre eixos.

Conclusões

5 Resultados e conclusão

- STM32L475 com DDA a 50 kHz entrega geração determinística de pulsos e malhas de controle estáveis.
- Malhas de posição e velocidade com encoders TMCS-28 e PI/PD inteiros rastreiam bem perfis trapezoidais típicos de CNC.
- Protocolo SPI com quadros fixos e CRC de 16 bits garante enlace robusto com a Raspberry Pi.
- Roteiro incremental com telemetria facilitou validar timers, SPI, encoders e PID.
- Modelo FOPDT e catálogo de ganhos PD viabilizaram sintonia sistemática da malha de velocidade, coerente com a simulação.

Trabalhos futuros

5 Resultados e conclusão

- Reduzir a dependência de múltiplos *polls* SPI por comando.
- Explorar reinicialização imediata do DMA assim que uma resposta estiver disponível.
- Avaliar envio de múltiplos comandos por ciclo SPI para melhor aproveitamento do canal.
- Implementar e avaliar outros modelos de controle baseados na correção preditiva do movimento dos motores.

Obrigado!

Perguntas?